

ЛАБОРАТОРНАЯ РАБОТА №7

Табулированные функции: Паттерны проектирования «Итератор» и «Фабричный метод». Механизм рефлексии.

**по курсу
Объектно-ориентированное программирование**

Группа 6204-010302D

Студент: Е.Д.Васильев.

**Преподаватель: Борисов Дмитрий
Сергеевич.**

Задание 1

Реализация паттерна «Итератор»

В рамках первого задания интерфейс **TabulatedFunction** был расширен наследованием от стандартного интерфейса **Iterable<FunctionPoint>**. Это позволило использовать объекты табулированных функций в циклах **for-each**. В классах **ArrayTabulatedFunction** и **LinkedListTabulatedFunction** был реализован метод **iterator()**, возвращающий итератор для последовательного обхода точек функции. Реализация выполнена с использованием анонимных внутренних классов. Для сохранения инкапсуляции метод **next()** возвращает копию объекта **FunctionPoint**, а не ссылку на внутренний элемент. Метод **remove()** при вызове выбрасывает исключение **UnsupportedOperationException**.

Изменения:

TabulatedFunction:

```
public interface TabulatedFunction extends Function, Cloneable,
Iterable<FunctionPoint> {

    Iterator<FunctionPoint> iterator();

}
```

ArrayTabulatedFunction:

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0;

        public boolean hasNext() {
            return index < pointsCount;
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }

            FunctionPoint point = new FunctionPoint(points[index]);
            index++;
            return point;
        }

        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}
```

LinkedListTabulatedFunction:

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {

        private FunctionNode node = head.next;

        public boolean hasNext() {
            return node != head;
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }

            FunctionPoint point = new FunctionPoint(node.point);

            node = node.next;

            return point;
        }

        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}
```

Задание 2

Реализация паттерна «Фабричный метод»

Для обеспечения гибкости создания объектов и возможности динамического переключения между реализациями (массив или связный список) был применен порождающий паттерн проектирования «Фабричный метод».

В пакете **functions** был описан интерфейс **TabulatedFunctionFactory**, объявляющий методы для создания экземпляров табулированных функций с различными параметрами.

В классах **ArrayTabulatedFunction** и **LinkedListTabulatedFunction** были реализованы вложенные статические классы-фабрики (**ArrayTabulatedFunctionFactory** и **LinkedListTabulatedFunctionFactory**), реализующие данный интерфейс.

В утилитарный класс **TabulatedFunctions** было добавлено статическое поле **factory**, хранящее ссылку на текущую используемую фабрику (по умолчанию — фабрика массивов), а также метод **setTabulatedFunctionFactory** для её замены. Метод **tabulate** был модифицирован: прямое создание объектов через оператор **new** заменено на вызов фабричного метода.

TabulatedFunctionFactory:

```
public interface TabulatedFunctionFactory {

    TabulatedFunction create(double leftX, double rightX, int pointsCount);
}
```

```

    TabulatedFunction create(double leftX, double rightX, double[] values);

    TabulatedFunction create(FunctionPoint[] points);
}

```

ArrayTabulatedFunction(вложенная фабрика):

```

public static class ArrayTabulatedFunctionFactory implements
TabulatedFunctionFactory {
    public TabulatedFunction create(double leftX, double rightX, int
pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction create(double leftX, double rightX, double[]
values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction create(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}

```

LinkedListTabulatedFunction (вложенная фабрика):

```

public static class LinkedListTabulatedFunctionFactory implements
TabulatedFunctionFactory {

    public TabulatedFunction create(double leftX, double rightX, int
pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction create(double leftX, double rightX, double[]
values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction create(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

Модификация класса TabulatedFunctions:

```

public final class TabulatedFunctions {
    // Метод для смены фабрики
    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory
newFactory) {
        factory = newFactory;
    }

    // Метод для получения текущей фабрики
    public static TabulatedFunctionFactory getFactory() {
        return factory;
    }

    // Методы-обертки для создания функций через фабрику
    public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return factory.create(leftX, rightX, pointsCount);
    }
}

```

```

}

public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
    return factory.create(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[]
points) {
    return factory.create(points);
}
}

```

Задание 3

Реализация «Рефлексии»

В утилитарный класс **TabulatedFunctions** были добавлены перегруженные методы **createTabulatedFunction** и добавлен перегруженный метод **tabulate**. В качестве аргумента они принимают объект типа **Class<? extends TabulatedFunction>**, который служит «шаблоном» для создания целевого объекта. Это позволяет методу работать с любыми реализациями интерфейса **TabulatedFunction**, без изменения исходного кода утилитного класса.

Алгоритм создания объекта через рефлексию:

1. С помощью метода **clazz.getConstructor(...)** происходит поиск публичного конструктора класса, сигнатура которого совпадает с передаваемыми параметрами.
2. С помощью метода **constructor.newInstance(...)** создается новый экземпляр класса.
3. Исключения, возникающие при работе с рефлексией перехватываются и оборачиваются в **IllegalArgumentException**.

TabulatedFunction:

```

public static TabulatedFunction tabulate(Class<? extends TabulatedFunction>
clazz, Function function, double leftX, double rightX, int pointsCount) {
    if (leftX < function.getLeftDomainBorder() || rightX >
function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Boundaries out of domain");
    }

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        if (i == pointsCount - 1) {
            x = rightX;
        }
        double y = function.getFunctionValue(x);
        points[i] = new FunctionPoint(x, y);
    }

    // Вызываем рефлексивный метод создания

```

```
        return createTabulatedFunction(clazz, points);
    }
}
```

Перегруженные createTabulatedFunction:

```
public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> clazz, FunctionPoint[] points) {
    try {
        // Ищем конструктор, принимающий массив FunctionPoint[]
        Constructor<? extends TabulatedFunction> constructor =
clazz.getConstructor(FunctionPoint[].class);
        // Создаем новый объект
        return constructor.newInstance(new Object[]{points});
    } catch (Exception e) {
        // Ловим любую ошибку рефлексии
        throw new IllegalArgumentException(e);
    }
}

// Создание функции через класс и параметры (leftX, rightX, pointsCount)
public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> clazz, double leftX, double rightX, int pointsCount) {
    try {
        // Ищем конструктор (double, double, int)
        Constructor<? extends TabulatedFunction> constructor =
clazz.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

// Создание функции через класс и параметры (leftX, rightX, values)
public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> clazz, double leftX, double rightX, double[] values) {
    try {
        // Ищем конструктор (double, double, double[])
        Constructor<? extends TabulatedFunction> constructor =
clazz.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}
```

Тест:

=====

ЗАДАНИЕ 1: ПРОВЕРКА ИТЕРАТОРА

=====

Создаем функцию экспоненты (Exp)

Табулируем функцию на отрезке [0, 5] (6 точек)

Функция создана: ArrayTabulatedFunction

Запускаем цикл for-each для перебора точек:

Точка: (0,0; 1,0)

Точка: (1,0; 2,7)

Точка: (2,0; 7,4)

Точка: (3,0; 20,1)

Точка: (4,0; 54,6)

Точка: (5,0; 148,4)

Итерация завершена успешно.

ЗАДАНИЕ 2: ПРОВЕРКА ФАБРИК

Табулируем Cos с фабрикой о по умолчанию

Результат: создан объект класса ArrayTabulatedFunction

Меняем фабрику на LinkedList

Фабрика установлена.

Табулируем Cos снова

Результат: создан объект класса LinkedListTabulatedFunction

Меняем фабрику обратно на Array

Фабрика установлена.

Табулируем Cos в третий раз

Результат: создан объект класса ArrayTabulatedFunction

ЗАДАНИЕ 3: ПРОВЕРКА РЕФЛЕКСИИ

Создаем ArrayTabulatedFunction через рефлексия (по границам и количеству)

Класс объекта: ArrayTabulatedFunction

Содержимое: {(0,0; 0,0), (5,0; 0,0), (10,0; 0,0)}

Создаем ArrayTabulatedFunction через рефлексия (по массивам значений)

Класс объекта: ArrayTabulatedFunction

Содержимое: {(0,0; 0,0), (10,0; 10,0)}

Создаем LinkedListTabulatedFunction через рефлексия (по массиву точек)

Класс объекта: LinkedListTabulatedFunction

Содержимое: {(0,0; 0,0), (10,0; 10,0)}

Табулируем функцию Sin через рефлексия (хотим получить LinkedList)

Класс объекта: LinkedListTabulatedFunction

Содержимое: {(0,0; 0,0), (0,3; 0,3), (0,6; 0,6), (0,9; 0,8), (1,3; 1,0), (1,6; 1,0), (1,9; 1,0), (2,2; 0,8), (2,5; 0,6), (2,8; 0,3), (3,1; 0,0)}